

Exercise Sheet 2

Lab Sessions: April 30 / May 8, 2026

1 Preparation

For this exercise, we provide input files and skeleton code through the [RepEng FIMGit Repository](#). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`

2 Dealing with Randomness

2.1 Data Generation: The pplease Script

Implement a Python command-line script called `pplease.py` that makes an input text “polite” by randomly inserting the phrase “, please” at the end of sentences, followed by the respective punctuation mark. For each sentence, there should be a 50% chance of appending “, please”. Thus, it is possible that the phrase is appended to all, some, or none of the sentences. The script should read text from `stdin` and write the output to `stdout`.

Assume you have a file `water.txt` with the following content:

```
I am thirsty. My glass is empty.  
Can you pass me the water?
```

Executing the script may look like this:

Terminal Session

bash

```
user@host$ cat water.txt | python pplease.py  
I am thirsty, please. My glass is empty.  
Can you pass me the water?
```

`cat` is a command line tool that prints a file’s content. The pipe (`|`) redirects the `stdout` of one command to become the `stdin` of the next, effectively chaining them together.

Note: If you use Large Language Models (LLMs) to assist you in writing this code, ensure that you understand the generated code.

2.2 Data Analysis

Produce a Python command-line script called `pplease_stats.py` to analyze the data generated by the `pplease.py` script from the previous task. This script should compute the minimum, maximum, and median amount of characters (including spaces and punctuation marks) per sentence for both the original input and its corresponding “polite” output.

The script should accept the input and output data via the command line, by taking file paths as arguments, and print the analysis results as a table on the command line.

Assume the input shown in Exercise 2.1 is stored in the file `water.txt`. Then, executing the `pplease.py` and `pplease_stats.py` scripts may look like this:

Terminal Session			bash
<pre>user@host\$ python pplease.py < water.txt > water_polite.txt</pre>			
<pre>user@host\$./pplease_stats.py water.txt water_polite.txt</pre>			
Metric	Original Sentences	Polite Sentences	
Min Length	13	13	
Max Length	45	53	
Median Length	29.0	33.0	

The first command redirects the file content into the `pplease.py` script (using `<`) and then redirects the output to `water_polite.txt` (using `>`). Alternatively, you could also use the following command: `cat water.txt | python pplease.py > water_polite.txt`. The computed statistics may, of course, deviate due to the nondeterministic nature of the `pplease.py` script.

Note: If you use Large Language Models (LLMs) to assist you in writing this code, ensure that you understand the generated code.

2.3 Repeatability

Our current experiment generates nondeterministic output. What modifications are required to ensure that the experiment is repeatable?

2.3.1 Implementing and Testing Repeatability Changes

Implement the necessary modifications you identified and test them with the script provided in the RepEng FIMGit Repository: `run_experiment.sh` (on Linux/Mac) or `run_experiment.ps1` (on Windows Powershell).

The script executes `pplease.py` and `pplease_stats.py` a specified number of times and compares the obtained results to the first run. It takes two arguments: the path to the input file, and the number of runs to execute.

The examples below show how to call the script, using the file `recipe.txt` as input and performing 10 runs. It also shows the expected output after making the experiment repeatable.

Notes:

- In the initial table, you may observe different numbers in the column “Polite Sentences” than shown below, but subsequent runs should produce the same result as your first run.
- Make sure that `pplease.py` and `pplease_stats.py` are located in the same directory as `run_experiment.sh`.
- On Windows, replace `./run_experiment.sh` with `./run_experiment.ps1`.

Terminal Session

bash

```
user@host$ ./run_experiment.sh recipe.txt 10
=== Run 1 (baseline) ===
Metric                Original Sentences    Polite Sentences
Min Length            83                   83
Max Length            227                  235
Median Length         102.5                110.5
--- Run 2: identical to run 1 ---
--- Run 3: identical to run 1 ---
--- Run 4: identical to run 1 ---
--- Run 5: identical to run 1 ---
--- Run 6: identical to run 1 ---
--- Run 7: identical to run 1 ---
--- Run 8: identical to run 1 ---
--- Run 9: identical to run 1 ---
--- Run 10: identical to run 1 ---
```

3 Reproducibility Package

Prepare the experiment so that a third party can easily *reproduce* your experiment. Build a fully automated reproducibility package by extending the Dockerfile skeleton below.

To simplify writing the Dockerfile, place all required files (`recipe.txt`, `pplease.py`, `pplease_stats.py`, and `run_experiment.sh`) in the same directory as the Dockerfile.

Dockerfile

```
# TODO: Add missing metadata and setup steps
FROM ubuntu:???

# Install packages
# TODO: add required packages
RUN apt-get update && apt-get install -y \
    ???

# Change working directory to /app
WORKDIR /app

# Copy files from the host to the directory /app inside the container
# TODO: Specify the files to be copied
COPY ??? /app/

# Make sure the script is executable and has the correct line endings
RUN chmod +x /app/run_experiment.sh && \
    sed -i 's/\r$//' /app/run_experiment.sh

# Call run_experiment.sh when running the container
CMD ["/app/run_experiment.sh", "recipe.txt", "10"]
```

Your Dockerfile must adhere to the following checklist of **required** items:

- The first 1–2 lines document the purpose of the package.
- State who owns the copyright for this code.
- Specify the license using an SPDX license identifier¹.
- Start off of a recent long-term maintained Ubuntu distribution (i.e., LTS).
- Declare who is the maintainer of the code using
`LABEL org.opencontainers.image.authors=<author-email>`.
- When installing packages, sort them alphabetically.
- Have no “dead code” in the recipe (no commented-out instructions).

Additionally, you may implement the following **optional** point:

- Pin the exact versions of the installed packages to avoid future dependency breakage.

¹SPDX license identifier offer a concise and standardized format to specify the license of code. For more information see <https://spdx.org/licenses/>

3.1 Testing your Dockerfile

After completing your Dockerfile, build the Docker image and run the container:

```
Terminal Session bash  
user@host$ docker build -t repeng_lab2 .  
user@host$ docker run --rm repeng_lab2  
=== Run 1 (baseline) ===  
Metric                Original Sentences    Polite Sentences  
Min Length            83                  83  
Max Length            227                  235  
Median Length         102.5              110.5  
--- Run 2: identical to run 1 ---  
--- Run 3: identical to run 1 ---  
--- Run 4: identical to run 1 ---  
--- Run 5: identical to run 1 ---  
--- Run 6: identical to run 1 ---  
--- Run 7: identical to run 1 ---  
--- Run 8: identical to run 1 ---  
--- Run 9: identical to run 1 ---  
--- Run 10: identical to run 1 ---
```

Notes:

- In the initial output table, you may observe different numbers in the column “Polite Sentences” than shown above, but subsequent runs should produce the same result as your first run.
- The `docker run` flag `--rm` tells Docker to automatically remove the container once it stops running. Since the container in this scenario only performs a short computation and directly prints the result, adding `--rm` is a good practice to ensure that stopped containers do not clutter your system.

4 Dockerfile Review (Antwort-Wahl-Verfahren)

In this task, exactly one answer is correct. Mark the correct answer.

Consider the following Dockerfile and the checklist from Exercise 3.

Dockerfile

```
1 # Builds the environment for our text analysis
2 # Copyright 2026, John Smith
3 # SPDX-License-Identifier: MIT
4 FROM ubuntu:latest
5
6 # maintainer: john.smith@example.com
7
8 RUN apt-get update && apt-get install -y \
9     python3-pip \
10    python3 \
11    curl \
12    git
13
14 # Copy all contents of the current directory on the host
15 # to /app in the container
16 COPY . /app
17 # Change working directory in the container to /app
18 WORKDIR /app
19
20 # chmod +x doall.sh
```

Evaluate the Dockerfile against the **required** items on the checklist.

1. Which line violates the **required** items?

- ☐ Line 2 ☐ Line 3 ☐ Line 4 ☐ None of these Options

2. Which line violates the **required** items?

- ☐ Line 6 ☐ Line 16 ☐ Line 18 ☐ None of these Options

3. Which line should be removed in order to improve the Dockerfile?

- ☐ Line 1 ☐ Line 3 ☐ Line 20 ☐ None of these Options

4. After fixing the issues identified above, are there still violations to the checklist of **required** items?

- ☐ Yes ☐ No

5. After fixing all issues identified above, can we guarantee that we will build the exact same Docker image in the long term (i.e., in several months/years)?

- ☐ Yes ☐ No